

AI Form & Vault — Knowledge & Connections Roadmap

This maps every phase of your roadmap to three things: how you'd explain it simply, what you actually need to *understand* technically, and what *connections* (accounts, APIs, services) you need to set up. Read it top to bottom once, then come back to each section when you start that phase.

Before Phase 1: Foundational Knowledge

You need a baseline in four areas before touching any phase-specific work.

Mobile development basics. Android app structure (Activities, Fragments or Jetpack Compose if you're going modern UI), the Android permission model (camera, storage), and how Gradle/build configs work. If you've never built an Android app, this alone is a few weeks of learning.

Backend/API literacy. REST APIs, JSON, OAuth2 (this underlies Google Sign-In and most third-party auth), and how API keys/tokens work. You don't need to build a backend yet, but you need to understand how an app talks to a server.

Data privacy basics. You're handling Aadhaar, PAN, and passport data — some of the most sensitive PII that exists. Before writing code, understand at a high level what regulations apply (in India, the Digital Personal Data Protection Act; if you ever have non-Indian users, GDPR-style principles). This isn't optional homework — it shapes how you design storage and consent from day one, and it's the first question any serious user, investor, or partner will ask you.

Account setup. Create a Firebase project, a Google Cloud Console project (for OAuth credentials), and get API access to Gemini (Google AI Studio) or OpenAI. Do this now so you're not blocked later.

Phase 1: Foundation (MVP)

Explain it simply: "We're building the basic skeleton — let a user sign in, take a photo of a document, and store it securely. No intelligence yet, just proving people will use it."

What you need to understand:

- Google Sign-In integration (Firebase Auth handles most of this, but you need to understand the OAuth consent flow underneath it)
- Camera/gallery intents in Android — how to capture or pick an image and get a usable file

- OCR fundamentals: the difference between on-device OCR (Google's ML Kit Text Recognition, free, fast, works offline) and Tesseract (more configurable, better for some scripts, slower). You don't need to understand OCR internals, but you need to know their tradeoffs to pick correctly.
- Room (Android's local database library) — how to define entities, DAOs, and queries
- Basic encryption-at-rest concepts — at minimum, SQLCipher or Android's EncryptedFile/Keystore APIs, since you're storing ID documents locally

Connections you'll need:

- Firebase project (Authentication + Firestore or just Firebase Auth if storage stays local for now)
- Google Cloud Console OAuth client ID (required for Google Sign-In)
- No external AI API needed yet — this phase is intentionally "dumb"

Key thing to validate: Can you reliably get clean OCR text out of real photos of one document type (start with just Aadhaar)? If OCR quality is poor here, everything downstream suffers.

Phase 2: Document Intelligence

Explain it simply: "Now the app doesn't just store a photo — it reads it and understands what it is: a name, a date of birth, a document type — automatically, with no manual typing."

What you need to understand:

- Prompt engineering for structured extraction — how to instruct an LLM to return clean JSON (e.g., `{"name": "...", "dob": "...", "doc_type": "..."}`) instead of conversational text
- LLM API usage: request/response format, token limits, cost per call, and crucially, latency (users won't wait 10 seconds per document)
- Basic NLP concepts: named entity recognition, document classification — you don't need to build these from scratch since the LLM does it, but you need to understand *why* it can fail (handwriting, glare, regional scripts, low resolution)
- Image preprocessing — deskewing, contrast adjustment, cropping — often improves OCR accuracy more than switching OCR engines does
- Designing a confidence/fallback flow: what happens when extraction is wrong or incomplete? (This is a UX problem as much as a technical one.)

Connections you'll need:

- Gemini API (Google AI Studio) or OpenAI API key, with billing enabled
- Decide here: does extraction happen on-device (privacy-friendly, limited model power) or via cloud API call (more powerful, but you're sending ID photos/text to a third party — this needs to be disclosed clearly and ideally redacted/minimized before sending)

Key thing to validate: Extraction accuracy across multiple document types, and whether your privacy story holds up once data starts leaving the device.

Phase 3: Relationship & Context

Explain it simply: "The app starts understanding *who* is in your documents and how they relate to you — recognizing that a name appearing across different documents might be your sister, spouse, or parent — and it asks you to confirm rather than assuming."

What you need to understand:

- Graph data modeling — nodes (people) and edges (relationships), even if you implement this as relational tables rather than a dedicated graph database initially
- Entity resolution / identity matching — the genuinely hard problem of deciding whether "Priya Sharma" in two different documents is the same person
- Confirmation-based UX design — how to ask a user "Is this your sister?" in a way that feels helpful, not invasive or repetitive
- When to introduce a real graph database (like Neo4j) versus when SQLite/Room with adjacency-list tables is good enough — for an MVP-stage identity graph, you likely don't need a dedicated graph DB yet

Connections you'll need:

- None new necessarily — this can run on the same LLM API from Phase 2 (asking it to extract relationships from context) plus your existing local storage. A dedicated Graph DB connection only becomes necessary once the relationship data outgrows what relational queries can handle cleanly.

Key thing to validate: Whether users actually find relationship-mapping useful, or whether it feels like overreach. This phase has real "creepy app" risk if not handled with visible, simple consent.

Phase 4: Smart Features

Explain it simply: "Now the vault becomes genuinely useful day-to-day — you can search your documents in plain English, snap a photo of any blank form and have it filled automatically, and the app suggests autofill anywhere on your phone."

What you need to understand:

- Semantic search / vector embeddings — to support natural language queries like "find my passport" or "when does my insurance expire," you need to embed document text and query against it (not just keyword match)
- The Android AutofillService API — this is a real, separate platform API with its own learning curve: how to register as an autofill provider, how to detect form fields in other apps, and Android's restrictions around what autofill services can access
- Field detection — recognizing what a blank field on a scanned or live form actually wants (name vs. date vs. address), which is a harder, fuzzier problem than it sounds
- Designing AI "suggestions" that are reversible and low-friction (users need to easily reject a bad autofill suggestion)

Connections you'll need:

- A local or lightweight vector index (libraries exist for on-device embedding search; for an MVP you may not need a hosted vector database)
- Deep integration with Android's Autofill Framework — no external account, but significant platform-specific study

Key thing to validate: Whether your AutofillService implementation actually feels reliable across different third-party apps — autofill UX breaks easily and inconsistently across Android versions and app types.

Phase 5: Complete Vision

Explain it simply: "The vault becomes a full personal AI assistant for your documents and identity — syncing across your devices, integrating with other apps and services, and proactively helping you, all while staying secure and under your control."

What you need to understand:

- End-to-end encryption design — not just encryption-at-rest, but how data stays encrypted in transit and how you manage keys such that even you (the developer) can't read user data

- Cross-device sync architecture — conflict resolution (what happens if a user edits the same document on two devices), sync protocols, and offline-first design
- Security auditing fundamentals — at this stage you're a real target; you need to understand common mobile app vulnerabilities (insecure storage, improper certificate pinning, etc.) or budget for a professional security audit
- Third-party integration patterns — OAuth scopes, API partnerships, what it means to let another company's app request data from your vault
- AI agentic workflows — multi-step planning where an LLM doesn't just extract data but takes actions (e.g., "renew my passport" triggers a multi-step flow), including tool-use patterns and guardrails against the AI taking wrong actions autonomously

Connections you'll need:

- A real cloud sync backend (Firebase can scale here, or you may need a custom backend)
 - Partnership/API agreements with third-party services you integrate with
 - Possibly a third-party security audit firm
-

Cross-Cutting: System Architecture Thinking

Independent of any single phase, you need to understand *layered system design* — how your Input Sources (DigiLocker, camera, manual entry) feed an AI Processing Layer (OCR, classification, extraction), which populates a Knowledge Layer (profile, identity graph, search index), which powers an Application Layer (autofill, search, suggestions). Each layer should be replaceable without breaking the others — e.g., swapping ML Kit for Tesseract shouldn't require changes to your storage layer. This is the single most useful mental model to internalize early, because it's what lets you explain the whole system to someone in 30 seconds: "documents go in, get understood, get organized, become useful everywhere."

Full Connections Checklist

Connection	Needed From	Purpose
Firebase project	Phase 1	Auth, possibly cloud storage/sync later
Google Cloud Console OAuth	Phase 1	Google Sign-In
ML Kit / Tesseract OCR	Phase 1	Text extraction (libraries, no account needed)
Gemini or OpenAI API key	Phase 2	Structured data extraction, relationship inference

Connection	Needed From	Purpose
Vector search library/DB	Phase 4	Natural language document search
Android Autofill Framework	Phase 4	System-wide autofill
Cloud sync backend	Phase 5	Cross-device sync
Third-party API partners	Phase 5	Integrations
Security audit firm	Phase 5	Pre-launch hardening

Suggested Learning Order

Don't try to learn all of this at once — that's the fastest way to stall before writing code. A reasonable sequence:

1. Android basics + Firebase Auth (1-2 weeks if new to Android)
2. OCR (ML Kit) + Room DB (1 week) — this gets you to a working Phase 1 prototype
3. LLM API usage + prompt engineering for structured JSON (3-5 days) — unlocks Phase 2
4. Everything else (graph modeling, autofill API, vector search) — learn just-in-time, as you reach each phase, rather than upfront

The honest version of "understanding everything before explaining it to anyone" is: understand Phase 1 and 2 deeply (you'll actually build these soon), and understand Phases 3-5 at the conceptual level mapped above (enough to explain the vision and defend the architecture, without yet needing implementation-level mastery).